

ساختار داده‌ها و الگوریتم‌ها

جلسه دوم – تحلیل الگوریتم‌ها: مرتب‌سازی درجی

دکتر شهرام گلزاری

پاییز ۱۴۰۵

- چرا و با چه هدفی الگوریتم‌ها را تحلیل می‌کنیم؟
- مرتب‌سازی درجی (Insertion Sort): ایده، شبه‌کد و اجرا
- چرا زمان‌سنجی کد کافی نیست؟
- مدل محاسباتی RAM: روی چه ماشینی داریم حساب می‌کنیم؟
- تحلیل دقیق مرتب‌سازی درجی: بهترین حالت، بدترین حالت و حالت میانگین

چرا الگوریتم‌ها را تحلیل می‌کنیم؟

تحلیل الگوریتم‌ها با هدف‌های زیر انجام می‌شود:

- بررسی و پیش‌بینی زمان اجرا و میزان حافظه‌ی مصرفی یک الگوریتم، **قبل از پیاده‌سازی**
- مقایسه‌ی الگوریتم‌های مختلف برای حل یک مسئله از نظر میزان کارایی

زمان اجرای یک برنامه و مقدار حافظه‌ی مصرفی آن، برای ورودی‌های مختلف، میزان خوبی برای سنجش کارایی الگوریتم استفاده شده در آن برنامه است.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۵۵

عوامل زیر در زمان اجرای یک برنامه بر روی یک کامپیوتر مؤثرند:

۱. سرعت پردازنده‌ی کامپیوتر،
۲. نوع کامپایلر یا زبان برنامه‌نویسی استفاده‌شده،
۳. اندازه‌ی داده‌ی ورودی مسئله،
۴. ترکیب یا ساختار داده‌های ورودی،
۵. پیچیدگی الگوریتم استفاده‌شده در برنامه، و
۶. عوامل دیگر که وابسته به ورودی نیستند و تأثیر خطی در زمان اجرای برنامه دارند.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۵۶

- سرعت پردازنده، نوع کامپایلر استفاده شده و عوامل مشابه، به صورت **خطی و با ضریب ثابت** در زمان اجرا برنامه‌ها موثرند.
- اگر پردازنده را دو برابر سریع‌تر کنیم و دیگر عوامل تغییر نکنند، انتظار داریم که در بهترین حالت زمان اجرای برنامه بر روی کامپیوتر جدید **نصف** شود؛ انتخاب نوع کامپایلر هم تاثیری مشابه سرعت پردازنده دارد.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۵۶

اندازه‌ی ورودی: هر قدر ورودی بزرگ‌تر باشد، اجرای برنامه هم با آن ورودی زمان بیش‌تری به طول خواهد کشید. تابع پیچیدگی الگوریتم استفاده شده، رابطه‌ی این دو مقدار را معین می‌کند.

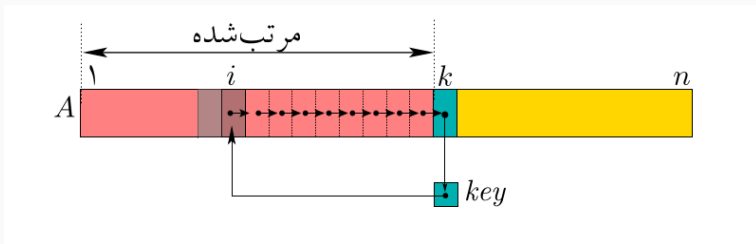
ترکیب ورودی: این فقط اندازه‌ی ورودی نیست که در زمان اجرا مؤثر است، بلکه ساختار یا ترکیب اجزای داده‌ی ورودی هم می‌تواند شرایطی را ایجاد کند که برنامه مثلاً خیلی کند، یا برای همان تعداد داده با ترکیب متفاوت، سریع جواب دهد. مثالی از ترکیب داده‌ی ورودی، مثلاً مرتب‌بودن یا نبودن، برعکس یا تقریباً مرتب‌بودن مجموعه‌ی داده در یک الگوریتم مرتب‌سازی است.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۵۶

از مفهوم ترکیب داده‌ی ورودی نتیجه می‌گیریم که بهتر است در صورت امکان، تابع زمان اجرای یک برنامه (یا پیچیدگی الگوریتم مورد استفاده) را هم برای بهترین و هم برای بدترین حالت ورودی (که برای یک اندازه‌ی ورودی ثابت، به ترتیب کم‌ترین و بیش‌ترین زمان اجرا را نشان می‌دهد) به دست آوریم. همچنین با فرض داشتن یک تابع توزیع از ترکیب ورودی، می‌توانیم تابع زمان اجرای قابل انتظار (یا پیچیدگی در حالت میانگین) را هم به دست آوریم. اگر تابع پیچیدگی را در حالت‌های مختلفی که گفته شد به دست آوریم، دانش ما از الگوریتم مورد استفاده بیش‌تر می‌شود.

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۵۷

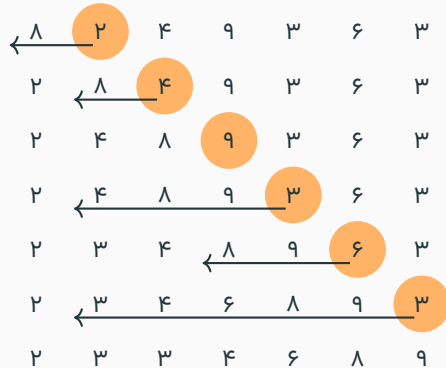
مرتب‌سازی درجی (Insertion Sort)



در مرحله‌ی k ام، زیرآرایه‌ی سمت چپ **مرتب شده** است و عنصر جدید (key) در جای درست خود درج می‌شود؛ عناصر بزرگ‌تر یک خانه به جلو شیفت می‌خورند.

شکل ۱-۳، داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۵۸

مثالی از مراحل اجرای مرتب‌سازی درجی



عنصر نارنجی در هر سطر، key آن مرحله است. توجه کنید در سطر سوم (درج ۹) هیچ جابه‌جایی‌ای لازم نیست.

بازترسیم شکل ۳-۲، داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۵۹

INSERTION-SORT(A, n)

```

1  for  $i = 2$  to  $n$ 
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$ .
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 

```

مدل محاسباتی RAM

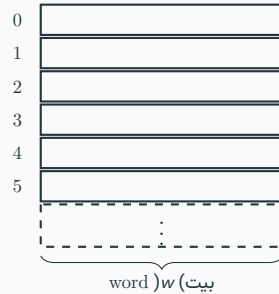
پیش از آن که بتوانیم الگوریتمی را تحلیل کنیم، به مدلی از فناوری ای که الگوریتم روی آن اجرا می شود نیاز داریم؛ شامل منابع آن فناوری و راهی برای بیان هزینه ی آن منابع.

CLRS در بیشتر کتاب، مدل **random-access machine (RAM)** را با یک پردازنده به عنوان فناوری پیاده سازی فرض می کند:

- دستورها یکی پس از دیگری اجرا می شوند و عملیات هم روند وجود ندارد.
- هر دستور به همان اندازه ی هر دستور دیگر زمان می برد و هر دسترسی به داده خواندن مقدار یک متغیر یا نوشتن در آن به همان اندازه ی هر دسترسی دیگر زمان می برد.

مدل RAM: حافظه به صورت یک آرایه‌ی بزرگ

- حافظه‌ی با دسترسی تصادفی، با یک آرایه‌ی بزرگ مدل می‌شود.
- توضیح بیشتر:



MIT 6.006 Fall 2011 Lecture note of session 2

تحليل مرتب سازی درجی

INSERTION-SORT(A, n)	$cost$	$times$
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 // Insert $A[i]$ into $A[1 : i - 1]$.	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n - 1)$$

بهترین حالت: آرایه‌ی از قبل مرتب

حتی برای ورودی‌هایی با اندازه‌ی مشخص، زمان اجرای یک الگوریتم می‌تواند به این بستگی داشته باشد که **کدام** ورودی آن اندازه داده شده است.

در INSERTION-SORT بهترین حالت وقتی رخ می‌دهد که آرایه **از قبل مرتب** باشد. در این حالت هر بار که خط ۵ اجرا می‌شود، مقدار key از پیش بزرگ‌تر یا مساوی همه‌ی مقادیر $A[1 : i - 1]$ است، پس حلقه‌ی `while` (خط‌های ۵ تا ۷) همیشه در همان نخستین اجرای خط ۵ خارج می‌شود. بنابراین $t_i = 1$ برای $i = 2, 3, \dots, n$

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8) n - (c_2 + c_4 + c_5 + c_8) \end{aligned} \quad (1.2)$$

این زمان اجرا را می‌توان به صورت $an + b$ نوشت؛ یعنی یک **تابع خطی** از n .

بدترین حالت وقتی پیش می‌آید که آرایه به **ترتیب معکوس** مرتب باشد، یعنی نزولی شروع شود. در این صورت رویه باید هر عنصر $A[i]$ را با تمام عناصر زیرآرایه‌ی مرتب‌شده‌ی $A[1 : i - 1]$ مقایسه کند، پس $t_i = i$ برای $i = 2, 3, \dots, n$.

با توجه به این‌که

$$\sum_{i=2}^n i = \left(\sum_{i=1}^n i \right) - 1 = \frac{n(n+1)}{2} - 1 \quad \text{و} \quad \sum_{i=2}^n (i-1) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

$$T(n) = c_1 n + c_2 (n-1) + c_3 (n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8 (n-1)$$

$$T(n) = \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n - (c_2 + c_4 + c_5 + c_8) \quad (2.2)$$

این زمان اجرای بدترین حالت را می‌توان به شکل $an^2 + bn + c$ برای ثابت‌های a ، b و c نوشت که باز هم به هزینه‌های دستورها (c_k) بستگی دارند:

$$c = -(c_2 + c_4 + c_5 + c_8) \quad b = c_1 + c_2 + c_4 + c_5/2 - c_6/2 - c_7/2 + c_8 \quad a = c_5/2 + c_6/2 + c_7/2$$

نتیجه

زمان اجرا در بدترین حالت یک **تابع درجه دوم** (quadratic function) از n است.

۱. بدترین حالت یک **کران بالا** برای زمان اجرا روی هر ورودی می‌دهد؛ تضمین می‌کند الگوریتم هرگز بیش از آن طول نمی‌کشد. لازم نیست حدس بزنیم و امیدوار باشیم بدتر نشود. این ویژگی به‌ویژه در محاسبات بی‌درنگ (real-time) مهم است، جایی که عملیات باید تا یک مهلت مشخص تمام شوند.
۲. برای برخی الگوریتم‌ها بدترین حالت **نسبتاً پرتکرار** است. مثلاً در جست‌وجوی یک پایگاه‌داده برای یافتن اطلاعات، بدترین حالت الگوریتم جست‌وجو اغلب زمانی رخ می‌دهد که آن اطلاعات در پایگاه‌داده **موجود نباشد** و در برخی کاربردها جست‌وجو برای اطلاعات غایب می‌تواند پرتکرار باشد.
۳. حالت میانگین اغلب **تقریباً به بدی** بدترین حالت است.

فرض کنید مرتب‌سازی درجی را روی آرایه‌ای از n عدد تصادفی اجرا می‌کنیم. تعیین این‌که $A[i]$ کجای زیرآرایه‌ی $A[1 : i - 1]$ باید درج شود چقدر طول می‌کشد؟

- به‌طور متوسط، نیمی از عناصر $A[1 : i - 1]$ کوچک‌تر از $A[i]$ و نیم دیگر بزرگ‌تر از آن‌اند.
- پس به‌طور متوسط $A[i]$ تنها با نیمی از زیرآرایه‌ی $A[1 : i - 1]$ مقایسه می‌شود، یعنی $t_i \approx i/2$.
- در نتیجه، زمان اجرای حالت میانگین هم یک تابع درجه دوم از اندازه‌ی ورودی از آب درمی‌آید درست مثل زمان اجرای بدترین حالت.

منابع

- داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، انتشارات فاطمی – صفحه‌های ۵۵ تا ۶۵ (شکل‌های ۱-۳ و ۲-۳)
- اسلایدهای تدریس همین درس توسط دکتر قدسی در نیم‌سال اول ۱۳۸۸-۱۳۸۹
- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition – صفحه‌های ۱۹، ۲۶، ۲۷، ۳۰، ۳۱ و ۳۲
- MIT 6.006 — Introduction to Algorithms: Lecture 2 (Fall 2011)